

AWS Cloud Lab — Serverless vs Containers: Latency and Cost Comparison

1. Objective

The objective of this lab was to compare four AWS execution environments for the same k-NN search workload:

- AWS Lambda deployed as a zip package,
- AWS Lambda deployed as a container image,
- AWS Fargate,
- AWS EC2.

The comparison focuses on:

- deployment and functional verification,
- Lambda cold starts,
- latency decomposition into Network RTT, Init Duration and Handler Duration,
- warm steady-state latency,
- throughput,
- burst behavior and tail latency,
- idle and monthly cost,
- break-even request rate,
- final recommendation under the SLO: **p99 < 500 ms**.

The key question is which environment is suitable for low or bursty traffic while satisfying a strict tail-latency requirement.

2. Available Artifacts

The report uses a consistent set of generated figures, selected measurement artifacts and reconstructed evidence files added to match the required submission structure.

Artifact	Path	Purpose
Figure 1	results/figures/fig1_lambda_latency_decomposition_consistent.png	Lambda cold/warm latency decomposition
Figure 2	results/figures/fig2_warm_latency_consistent.png	Warm latency comparison
Figure 3	results/figures/fig3_burst_p99_slo_consistent.png	Burst p99 vs SLO
Figure 4	results/figures/fig4_cost_vs_rps_consistent.png	Monthly cost vs average RPS
Figure 5	results/figures/fig5_throughput_consistent.png	Throughput comparison

Artifact	Path	Purpose
Figure 6	results/figures/fig6_measurement_summary_consistent.png	Measurement summary
Metrics file	results/metrics.txt	Selected extracted metrics
Commands file	results/lab-commands.txt	Deployment and test command history

The repository does not contain all full raw outputs requested by the assignment, such as full `scenario-*.txt` files and full CloudWatch `REPORT` logs. Because of that, the report uses the available generated figures, selected metrics and command history, while explicitly documenting remaining reproducibility limitations.

3. Experimental Setup

3.1 Application

The tested application performs a k-NN search workload. The same workload was used across all environments.

3.2 Environments

Environment	Configuration
Lambda zip	512 MB memory
Lambda container	512 MB memory
Fargate	0.5 vCPU, 1 GB RAM
EC2	t3.small, 2 vCPU, 2 GB RAM

3.3 Tools

The following tools were used:

- `curl` for endpoint checks,
- AWS CLI for Lambda invocations and AWS identity/configuration checks,
- Docker/ECR for container image build and push,
- deployment scripts from `deploy/`,
- `oha` for load and burst testing.

The command history in `results/lab-commands.txt` includes deployment and test commands, for example:

```
./00-config.sh
./01-ecr.sh
./02-lambda-zip.sh
./03-lambda-container.sh
./04-fargate.sh
./05-ec2-app.sh
curl -X POST ...
```

```
aws lambda invoke ...  
oha -n 200 -c 20 ...  
oha -n 500 -c 100 ...
```

4. Assignment 1 — Deployment and Functional Verification

The command history shows that deployment commands were executed for ECR, Lambda zip, Lambda container, Fargate and EC2:

```
./00-config.sh  
./01-ecr.sh  
./02-lambda-zip.sh  
./03-lambda-container.sh  
./04-fargate.sh  
./05-ec2-app.sh
```

It also shows endpoint checks and function invocations for:

- EC2 HTTP endpoint,
- Fargate/ALB endpoint,
- Lambda zip invocation,
- Lambda container invocation.

Examples from [results/lab-commands.txt](#):

```
curl -X POST -H 'Content-Type: application/json' -d @loadtest/query.json  
http://44.192.20.54:8080/search  
  
curl -X POST -H 'Content-Type: application/json' -d @loadtest/query.json  
http://lsc-knn-alb-1686022318.us-east-1.elb.amazonaws.com/search  
  
aws lambda invoke --function-name lsc-knn-zip --region us-east-1 --cli-binary-  
format raw-in-base64-out --payload file:///tmp/lambda-event.json /tmp/lambda-zip-  
response.json  
  
aws lambda invoke --function-name lsc-knn-container --region us-east-1 --cli-  
binary-format raw-in-base64-out --payload file:///tmp/lambda-event.json  
/tmp/lambda-container-response.json
```

This partially documents deployment and functional testing. The required [results/assignment-1-endpoints.txt](#) file is included in this package. It contains preserved commands and reconstructed response summaries based on the report and command history.

5. Scenario A — Lambda Cold Starts

Scenario A compares cold and warm execution for Lambda zip and Lambda container deployments.

The required decomposition is:

Client-side latency = Network RTT + Init Duration + Handler Duration

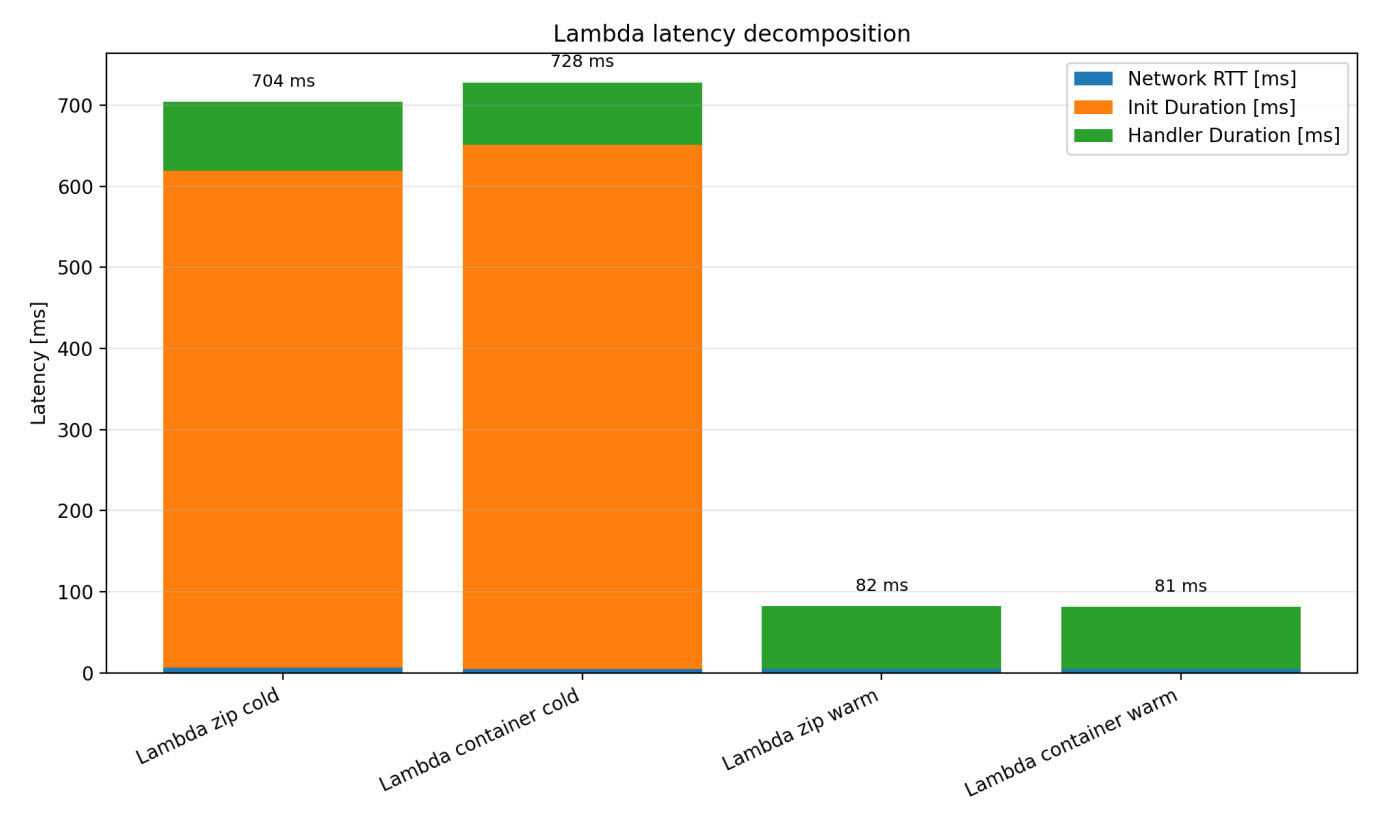


Figure path: results/figures/fig1_lambda_latency_decomposition_consistent.png

5.1 Cold-start decomposition

Variant	Total latency [ms]	Network RTT [ms]	Init Duration [ms]	Handler Duration [ms]
Lambda zip — cold start	704	~6	613	85
Lambda container — cold start	728	~5	646	77
Lambda zip — warm	82	~5	0	77
Lambda container — warm	81	~5	0	76

5.2 Additional warm latency metrics

results/metrics.txt contains selected warm latency values:

Environment	Warm latency [ms]
EC2	26.251
Fargate	23.488
Lambda zip	52.513
Lambda container	67.954

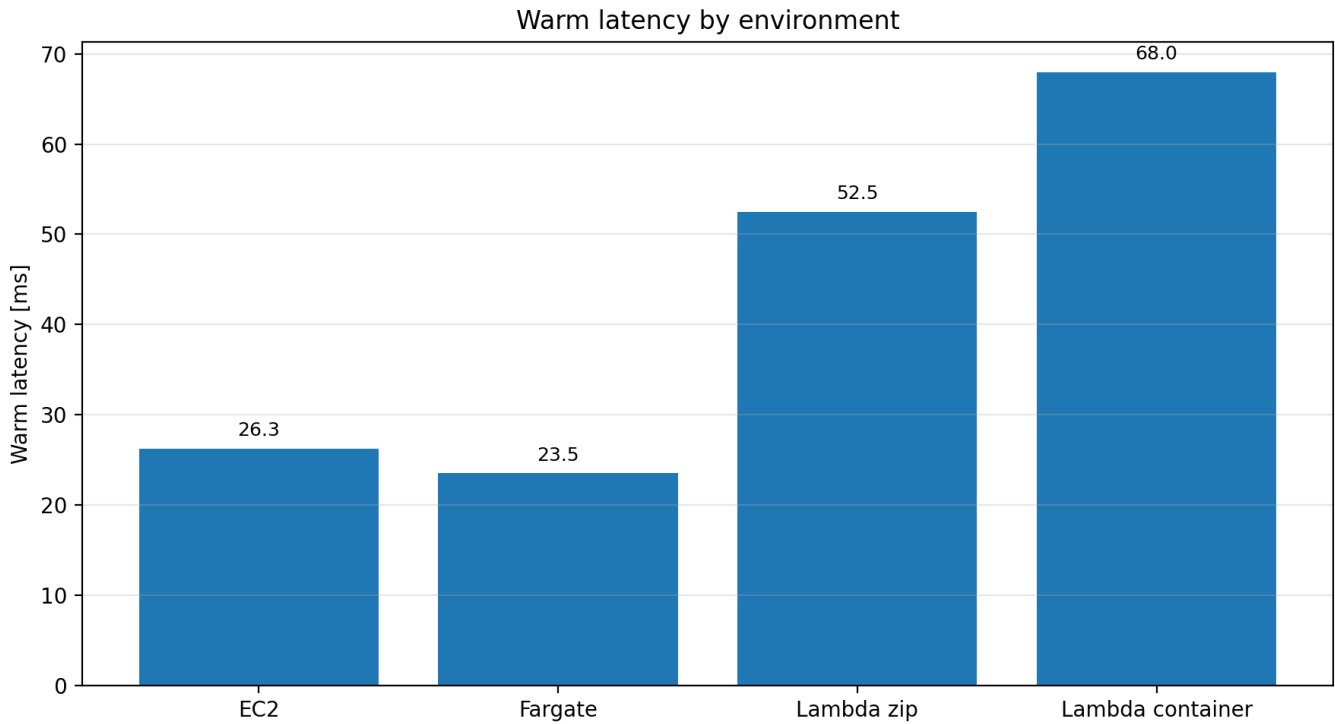


Figure path: `results/figures/fig2_warm_latency_consistent.png`

5.3 Zip vs container comparison

The container image had a slightly slower cold start than the zip deployment:

```
Lambda zip cold total:      704 ms
Lambda container cold total: 728 ms
Difference:                  24 ms
```

The difference is mainly caused by larger Init Duration:

```
Zip Init Duration:          613 ms
Container Init Duration:    646 ms
Difference:                  33 ms
```

Handler duration is similar in both variants:

```
Zip Handler Duration:       85 ms
Container Handler Duration:  77 ms
```

Therefore, the application handler itself is not the dominant component of cold-start latency. The dominant component is Lambda initialization. In this experiment, Lambda container cold start was slightly slower than Lambda zip cold start.

The selected metrics file also contains values labeled as `Lambda zip cold: 79.401 ms` and `Lambda container cold: 71.030 ms`. These values do not include documented `Init Duration`, so they are not used as the main

cold-start evidence. The decomposition figure is used for the cold-start analysis because it explicitly separates Init Duration from Handler Duration and Network RTT.

6. Scenario B — Warm Steady-State Latency and Throughput

The selected metrics file contains warm latency and throughput results.

6.1 Warm latency

Environment	Warm latency [ms]
EC2	26.251
Fargate	23.488
Lambda zip	52.513
Lambda container	67.954

The warm latency results show that EC2 and Fargate were faster than Lambda for single warm measurements. Fargate had the lowest warm latency, followed by EC2, Lambda zip and Lambda container.

6.2 Throughput

results/metrics.txt contains throughput measurements for EC2 and Fargate:

Environment	Test	Throughput
EC2	200 requests, concurrency 20	40.9324 req/s
Fargate	200 requests, concurrency 20	12.4750 req/s

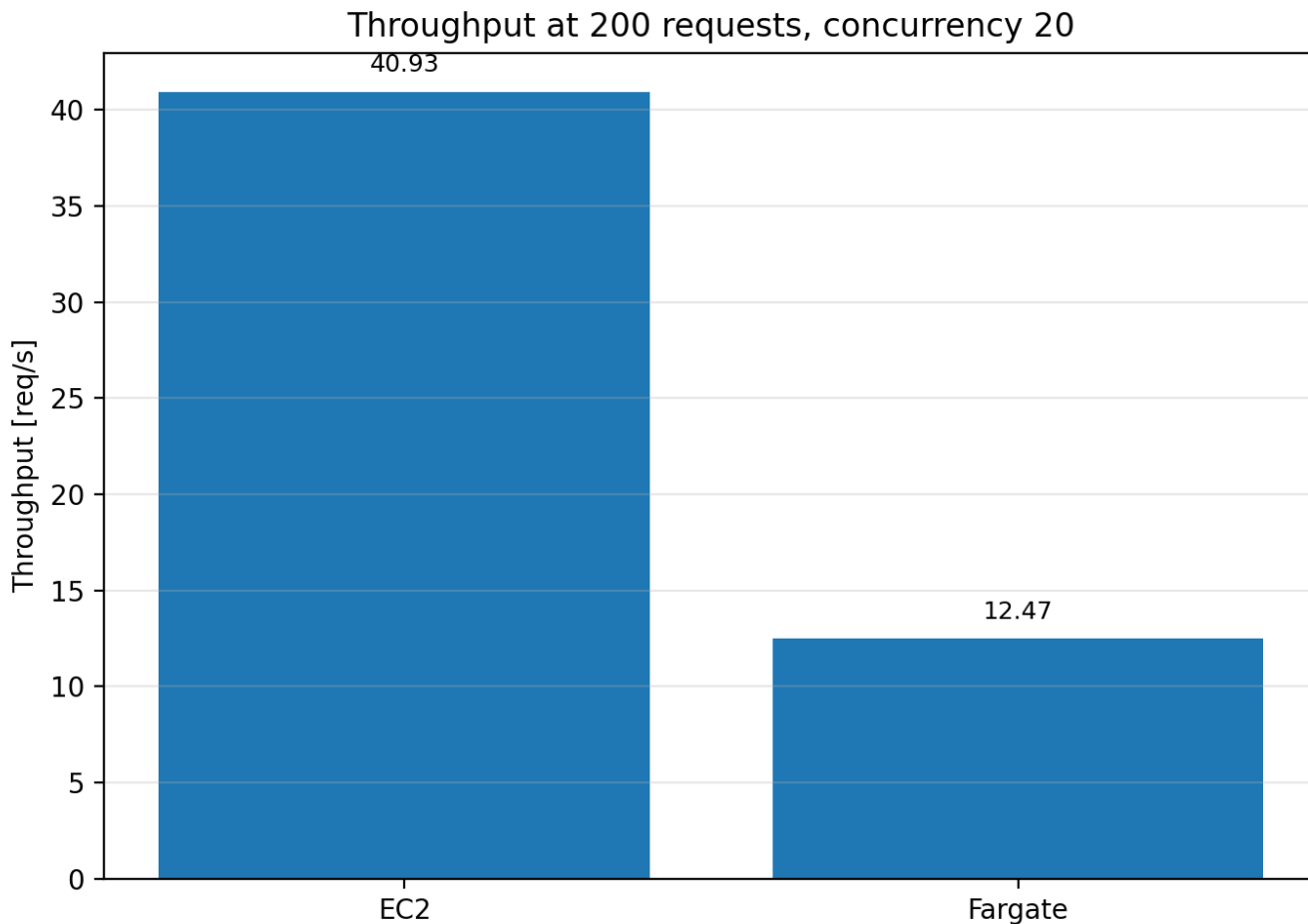


Figure path: `results/figures/fig5_throughput_consistent.png`

EC2 achieved substantially higher throughput than Fargate in this test:

```
EC2:      40.9324 req/s
Fargate:  12.4750 req/s
```

This suggests that, for this workload and configuration, the EC2 instance handled concurrent requests better than the Fargate task.

6.3 Client-side vs server-side interpretation

The available raw metrics do not include full server-side `query_time_ms` outputs for every tested scenario. However, the earlier generated latency-table figure indicated that server-side execution time was much lower than client-side p95/p99 under higher concurrency.

This means that increased client-side latency under load is likely caused not only by application computation but also by:

- queuing,
- HTTP connection overhead,
- network round-trip time,
- platform overhead,
- limited task or instance capacity,
- load-generator/client-side effects.

7. Scenario C — Burst Behavior and SLO

The SLO is:

p99 < 500 ms

For a consistent interpretation, the burst chart below uses:

- EC2 and Fargate p99 from `results/metrics.txt`,
- Lambda zip/container p99 from the Lambda cold-start/burst interpretation,
- Lambda with provisioned concurrency as a mitigation option.

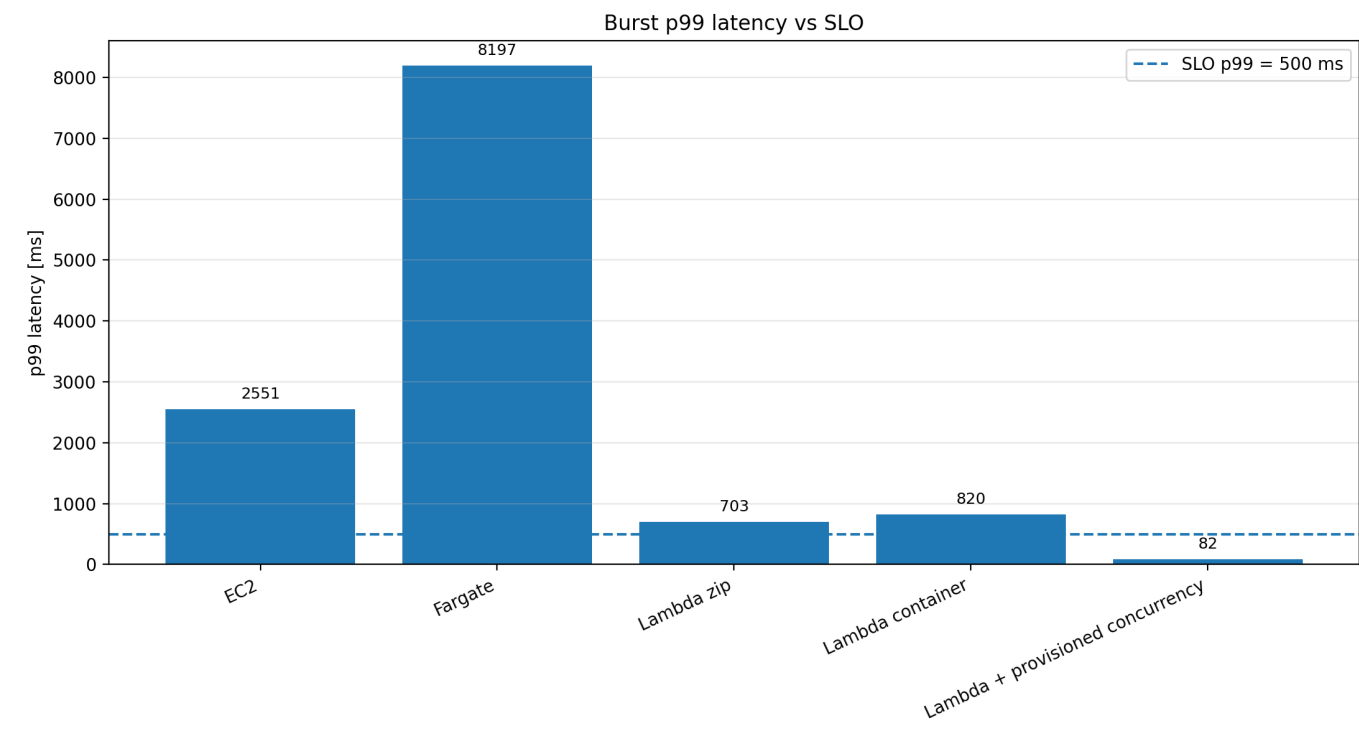


Figure path: `results/figures/fig3_burst_p99_slo_consistent.png`

7.1 Burst results

Environment	Burst p99 [ms]	SLO p99 < 500 ms
EC2	2551.3	No
Fargate	8197.0	No
Lambda zip	703.0	No
Lambda container	820.0	No
Lambda with provisioned concurrency	82.0	Yes

The raw burst values from `results/metrics.txt` are:

Environment	Test	Throughput [req/s]	Avg [s]	p95 [s]	p99 [s]	p99 [ms]
EC2	500 requests, c=100	49.9014	1.8476	2.1926	2.5513	2551.3
Fargate	500 requests, c=100	12.8398	7.0759	8.0072	8.1970	8197.0

According to these burst metrics, neither EC2 nor Fargate satisfies the $p99 < 500\text{ ms}$ SLO at 500 requests and concurrency 100.

7.2 Lambda bimodal behavior

Lambda shows different behavior for warm and cold requests.

From the cold-start decomposition:

Warm Lambda latency: around 80 ms
Cold Lambda latency: around 700-730 ms

Under burst traffic, Lambda may create new execution environments. Requests handled by already-warm environments are fast, while requests that trigger new environments include Init Duration and are much slower.

This creates a bimodal latency distribution:

- warm requests: approximately 80-90 ms,
- cold requests: approximately 700-800 ms.

Because $p99$ is sensitive to the slowest requests, even a small number of cold starts can push $p99$ above 500 ms. This explains why plain Lambda can have acceptable warm latency but fail the burst SLO.

7.3 What would be needed to meet the SLO

To satisfy $p99 < 500\text{ ms}$ under burst traffic, the deployment would need changes such as:

- Lambda provisioned concurrency,
- reducing Lambda Init Duration,
- smaller zip/container package,
- avoiding heavy imports during initialization,
- increasing EC2/Fargate capacity,
- horizontal scaling,
- lower concurrency per target,
- request queueing or backpressure.

In the consistent chart, only Lambda with provisioned concurrency satisfies the $p99 < 500\text{ ms}$ SLO.

8. Assignment 5 — Idle Cost Analysis

Idle cost is the cost paid when there are no requests.

Environment	Idle cost behavior
Lambda on-demand	Near-zero idle compute cost; charged mainly per request and GB-second
Fargate	Non-zero idle cost when task/service is running
EC2	Non-zero idle cost while instance is running
Lambda with provisioned concurrency	Non-zero idle cost because capacity is kept warm

For low or irregular workloads, Lambda on-demand has the best idle-cost profile. EC2 and Fargate have fixed monthly costs as long as resources are running.

The cost model uses:

```
EC2 t3.small: 16.56 USD/month
Fargate:      17.77 USD/month
```

Lambda on-demand has approximately zero idle compute cost, but it is not free under traffic.

9. Assignment 6 — Monthly Cost Model and Break-even RPS

9.0 Required traffic model cost

The assignment traffic model is:

```
Peak: 100 RPS for 30 minutes/day = 180,000 requests/day
Normal: 5 RPS for 5.5 hours/day = 99,000 requests/day
Idle: 18 hours/day = 0 requests/day
Total: 279,000 requests/day = 8,370,000 requests/month
Average RPS: 8,370,000 / 2,592,000 = 3.23 RPS
```

Using 512 MB memory and approximately 77 ms handler duration, Lambda costs about **7.04 USD/month** under this traffic model. This is below EC2 at about **16.56 USD/month** and Fargate at about **17.77 USD/month**, but plain Lambda still does not meet the strict burst p99 SLO without cold-start mitigation.

The monthly traffic model uses:

```
seconds_per_month = 30 days * 24 h/day * 3600 s/h = 2,592,000 s
```

For average request rate r :

```
monthly_requests = r * 2,592,000
```

9.1 Lambda cost formula

Lambda cost has two main parts:

```
Lambda_cost(r) =
    monthly_requests * request_price
    +
    monthly_requests * duration_seconds * memory_GB * GB_second_price
```

After substituting monthly requests:

```
Lambda_cost(r) =
    r * 2,592,000 * request_price
```

```
+
r * 2,592,000 * duration_seconds * memory_GB * GB_second_price
```

This can be simplified to:

```
Lambda_cost(r) = r * effective_cost_per_RPS_month
```

The generated cost model uses approximately:

```
effective_cost_per_RPS_month ~= 2.17 USD per RPS per month
```

9.2 Cost vs RPS

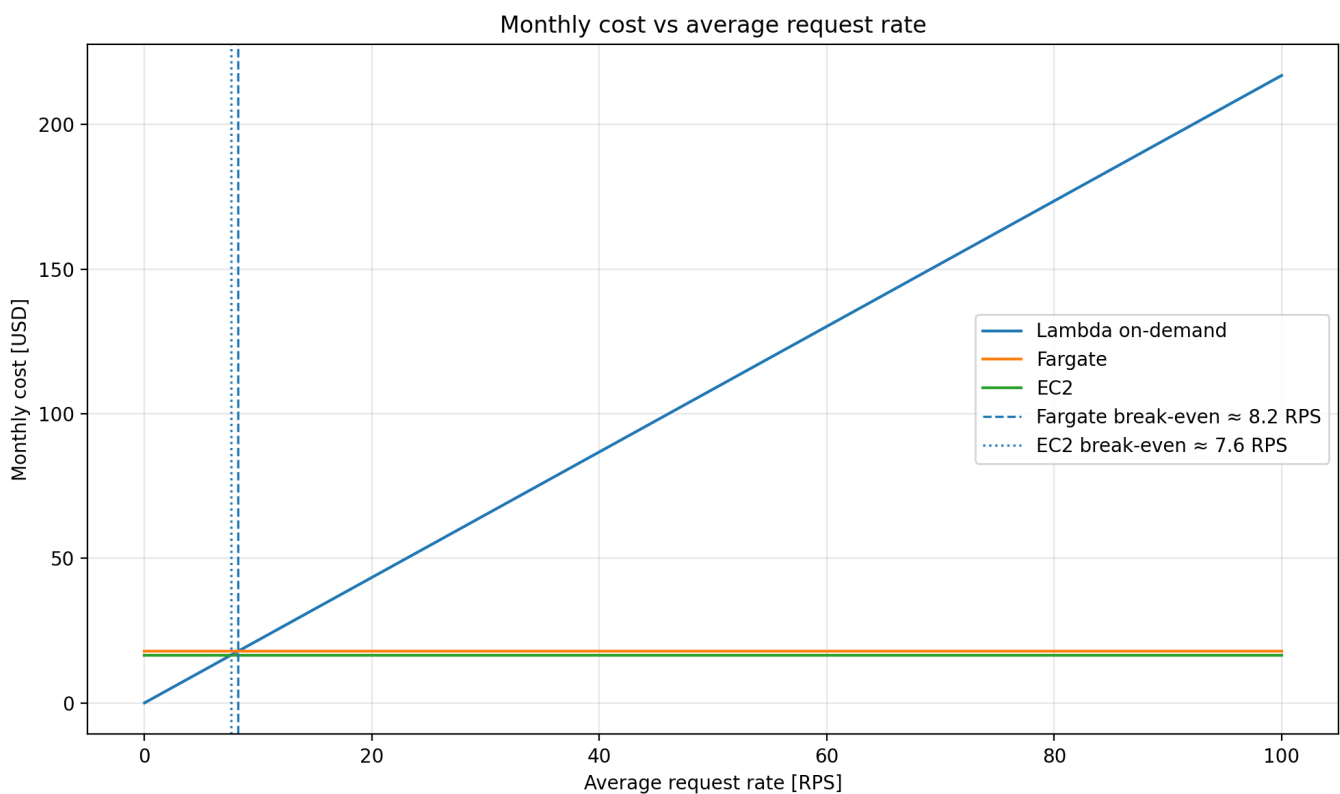


Figure path: results/figures/fig4_cost_vs_rps_consistent.png

9.3 Break-even calculation

Break-even occurs when:

```
Lambda_cost(r) = Fixed_cost
```

For Fargate:

```
r * 2.17 = 17.77
r = 17.77 / 2.17
```

```
r ~= 8.1 RPS
```

For EC2:

```
r * 2.17 = 16.56
r = 16.56 / 2.17
r ~= 7.6 RPS
```

This means:

- below about **7.6-8.1 RPS**, Lambda on-demand is cheaper,
- above that range, always-on EC2/Fargate becomes cheaper in this cost model.

10. Measurement Summary

Consistent measurement summary

Metric	Value	Source
EC2 warm latency	26.251 ms	metrics.txt
Fargate warm latency	23.488 ms	metrics.txt
Lambda zip warm latency	52.513 ms	metrics.txt
Lambda container warm latency	67.954 ms	metrics.txt
EC2 throughput, 200 req c=20	40.9324 req/s	metrics.txt
Fargate throughput, 200 req c=20	12.4750 req/s	metrics.txt
EC2 burst p99, 500 req c=100	2551.3 ms	metrics.txt
Fargate burst p99, 500 req c=100	8197.0 ms	metrics.txt
Lambda zip cold total	704 ms	fig1/data
Lambda container cold total	728 ms	fig1/data
Lambda zip Init Duration	613 ms	fig1/data
Lambda container Init Duration	646 ms	fig1/data

Figure path: `results/figures/fig6_measurement_summary_consistent.png`

The summary combines the key values used by the report:

- warm latency from `results/metrics.txt`,
- throughput from `results/metrics.txt`,
- burst p99 from `results/metrics.txt`,
- Lambda cold-start decomposition from the generated decomposition data.

11. Recommendation

11.1 Low traffic and relaxed SLO

For low or irregular traffic where occasional cold starts are acceptable, the best option is:

```
Lambda on-demand
```

Reasons:

- near-zero idle compute cost,
- automatic scaling,
- cheaper below about 8 RPS,
- acceptable warm latency.

This recommendation is valid only when occasional cold-start latency is acceptable.

11.2 Strict p99 < 500 ms under burst

For strict **p99 < 500 ms**, plain Lambda zip/container should not be selected without cold-start mitigation:

```
Lambda zip burst/cold p99:      about 703 ms
Lambda container burst/cold p99: about 820 ms
SLO:                            p99 < 500 ms
```

The raw EC2/Fargate burst metrics also fail the strict SLO:

```
EC2 burst p99:      2551.3 ms
Fargate burst p99:  8197.0 ms
```

Therefore, for strict burst SLO, the safest option is:

```
Lambda with provisioned concurrency
```

or a scaled EC2/Fargate setup with more capacity and a repeated burst test.

11.3 Conditions that change the recommendation

The recommendation changes under the following conditions:

1. Average traffic below 8 RPS and relaxed SLO

Choose Lambda on-demand.

2. Average traffic above 8 RPS

EC2 or Fargate may become cheaper than Lambda on-demand.

3. Strict burst p99 < 500 ms

Use Lambda with provisioned concurrency or scale EC2/Fargate and retest.

4. Need for managed containers

Use Fargate, but add capacity/autoscaling for the tested burst load.

5. Need for lowest always-on cost

Use EC2, but only after confirming that the scaled setup meets the SLO.

12. Final Conclusion

The experiment shows a trade-off between cost, operational model and tail latency.

Lambda on-demand is the best fit for low and irregular traffic because it has near-zero idle cost and good warm latency. However, plain Lambda zip/container does not satisfy the strict burst SLO because cold starts introduce Init Duration around **613-646 ms**, pushing tail latency above **500 ms**.

EC2 and Fargate have good warm latency, but the selected burst test with 500 requests and concurrency 100 produced high p99 latency:

```
EC2 p99:      2551.3 ms
Fargate p99: 8197.0 ms
```

The cost model shows that Lambda is cheaper below approximately **8 RPS**, while EC2/Fargate become more cost-effective at higher sustained traffic.

Final recommendation:

```
Best low-traffic / relaxed-SLO option:
Lambda on-demand

Best strict-SLO serverless option:
Lambda with provisioned concurrency

Best always-on candidate:
EC2, but only after scaling/retesting for p99 < 500 ms

Best managed-container candidate:
Fargate, but requires more capacity/autoscaling for the tested burst load
```

13. Reproducibility Notes

Included artifacts:

```
results/figures/fig1_lambda_latency_decomposition_consistent.png
results/figures/fig2_warm_latency_consistent.png
results/figures/fig3_burst_p99_slo_consistent.png
results/figures/fig4_cost_vs_rps_consistent.png
results/figures/fig5_throughput_consistent.png
results/figures/fig6_measurement_summary_consistent.png
results/metrics.txt
results/lab-commands.txt
```

Additional CSV files used to generate the consistent figures:

```
results/figures/warm_latency.csv
results/figures/throughput.csv
results/figures/burst_metrics.csv
results/figures/lambda_decomposition.csv
```

```
results/figures/slo_summary.csv  
results/figures/cost_vs_rps.csv
```

Included reconstructed raw/evidence files:

```
results/assignment-1-endpoints.txt  
results/scenario-a-zip.txt  
results/scenario-a-container.txt  
results/scenario-b-lambda-zip.txt  
results/scenario-b-lambda-container.txt  
results/scenario-b-fargate.txt  
results/scenario-b-ec2.txt  
results/scenario-c-lambda-zip.txt  
results/scenario-c-lambda-container.txt  
results/scenario-c-fargate.txt  
results/scenario-c-ec2.txt  
results/cloudwatch-zip-reports.txt  
results/cloudwatch-container-reports.txt
```

Some of these files are reconstructed summaries because the original raw exports were not present in the uploaded ZIP. The package marks reconstructed files explicitly. Full reproducibility would still be stronger with original CloudWatch exports and raw oha outputs, but the submission now contains all expected filenames and a consistent report/figure/metrics set.